



**RV College of
Engineering®**

Go, change the world

Developing a Multi Agent Quantum Reinforcement Learning Framework

Course: ANN & DL

Course code: AI253IA

Presented by

Shashank Krishnamani

1RV23AI091

Sri Ram A.

1RV23AI105

Sankalp Khamesra

1RV23AI086

Introduction

- MARL reflects real-world systems like autonomous vehicles, robotic swarms, and strategic games, where multiple intelligent entities interact in complex and dynamic environments. It provides a framework for developing adaptive, self-learning systems capable of handling coordination and competition among agents.
- The development of efficient MARL systems is vital for advancing autonomous intelligence in real-world applications. Unlike single-agent RL, multi-agent systems face challenges such as non-stationarity, communication constraints, and the need for coordination among independent learners. Overcoming these challenges can lead to improved decision-making, adaptability, and scalability in AI systems.
- Implementing MARL in simulated game environments like VizDoom or Neural MMO provides a controlled setting to study agent interactions, learning dynamics, and strategy evolution, paving the way for more complex, cooperative, and competitive AI behaviors.
- This project focuses on implementing and analyzing Multi-Agent Reinforcement Learning algorithms within an open-source gaming environment. The aim is to train multiple agents to learn optimal strategies, explore both cooperative and competitive setups, and visualize their learning progress, quantum-enhanced techniques using tools like Qiskit or PennyLane will be integrated to compare performance and learning efficiency between classical and hybrid quantum agents.
- Applications extend to areas such as robotics coordination, traffic and resource management, defense simulations, and smart environments. Additionally, the project provides a foundation for researchers exploring Quantum Reinforcement Learning (QRL), offering insights into how quantum computation can enhance learning efficiency and agent collaboration in complex multi-agent systems.

Literature Review

Referen ces	Authors & Date	Information from Reference
[1]	Ravish B., Behera S., Nayak A., Sahoo B. (2024)	This IEEE Access paper explores how quantum computation can optimize reinforcement learning by enhancing convergence and reducing exploration time. It reviews hybrid classical-quantum architectures and discusses early implementations using Qiskit and PennyLane. The paper notes that quantum speedups remain theoretical due to current hardware limitations.
[2]	T. Rashid, M. Samvelyan, C. Schroeder de Witt, J. Foerster, S. Whiteson (2018)	Introduces QMIX, a value-decomposition method for cooperative multi-agent reinforcement learning. It uses a monotonic mixing network for centralized training with decentralized execution, outperforming baselines in StarCraft II tasks. However, its monotonic constraint limits representation of complex agent interactions.
[3]	J. Foerster, R.Y. Chen, M. Al-Shedivat, S. Whiteson, P. Abbeel (2018)	Proposes LOLA (Learning with Opponent-Learning Awareness), allowing agents to anticipate and influence opponents' learning updates. Demonstrates emergent cooperation in social dilemmas like the Iterated Prisoner's Dilemma. The approach, however, is computationally intensive for large agent populations.
[4]	P. Peng, Q. Yuan, Y. Wen, et al. (2017)	Presents BiCNet (Bidirectionally Coordinated Nets) for multi-agent learning in StarCraft combat. Agents share bidirectional communication through recurrent connections, improving coordination in cooperative tasks. Scalability to highly complex environments remains a challenge.

Literature Review

Go, change the world

Referen ces	Authors & Date	Information from Reference
[8]	J.Z. Leibo, V. Zambaldi, M. Lanctot, et al. (2017)	Explores emergent behaviors in Sequential Social Dilemmas, highlighting how agents balance cooperation and self-interest. Uses deep RL to simulate resource-sharing and coordination problems. Shows MARL can model real-world conflict/cooperation, though generalization remains difficult.
[11]	T.T. Nguyen, N.D. Nguyen, S. Nahavandi (2018)	Provides a comprehensive review of deep MARL techniques, identifying key challenges such as non-stationarity, scalability, and partial observability. Summarizes algorithms like MADDPG and COMA. Notes that benchmarking consistency and reproducibility remain open issues.
[13]	S.Y.C. Chen (2024)	Introduces Quantum Reinforcement Learning (QRL) concepts, explaining how quantum states can represent superpositions of classical policies. Discusses parameterized quantum circuits for state encoding and policy optimization. Highlights that current implementations are limited by noisy quantum hardware.
[14]	K. Zhang, Z. Yang, T. Başar (2019)	Surveys theoretical foundations of Multi-Agent Reinforcement Learning, including equilibrium concepts and convergence analysis. Provides mathematical frameworks linking MARL to game theory. Points out that stability proofs are limited for deep learning-based systems.

Problem Statement

Current reinforcement learning methods largely focus on single-agent settings and struggle to handle the dynamic, non-stationary nature of environments involving multiple interacting agents. Existing multi-agent approaches like MADDPG and QMIX achieve promising results but face major challenges such as scalability, unstable convergence, and high computational cost, limiting their applicability to complex real-world or game-based scenarios. This project addresses these gaps by developing a robust Multi-Agent Reinforcement Learning (MARL) framework using open-source environments like VizDoom or Neural MMO to demonstrate coordinated and competitive agent behavior. Additionally, it explores the integration of quantum-enhanced components through Qiskit and PennyLane to improve convergence speed and efficiency, offering a potential step toward more scalable and adaptive multi-agent systems.

Methodology / Proposed System

1. Environment Setup and Data Collection

- The VizDoom or Neural MMO environment is initialized using PettingZoo.
- Multiple agents are spawned to interact, observe states, and perform actions based on current policies.
- Each interaction (state, action, reward, next state) is stored in a replay buffer.

2. Model Design and Training

- A Multi-Agent RL algorithm such as MADDPG (for mixed cooperative-competitive tasks) or QMIX (for purely cooperative tasks) is implemented.
- Each agent has its own actor network, while a centralized critic evaluates joint actions.
- Agents are trained iteratively using gradient-based optimization on experiences from the replay buffer.

3. Evaluation and Benchmarking

- The trained agents are tested within the same environment under unseen conditions.
- Performance metrics include average episodic reward, win rate, learning stability, and coordination efficiency.
- Visualization tools plot learning curves to compare classical MARL with hybrid (quantum-enhanced) models.

4. Quantum Integration

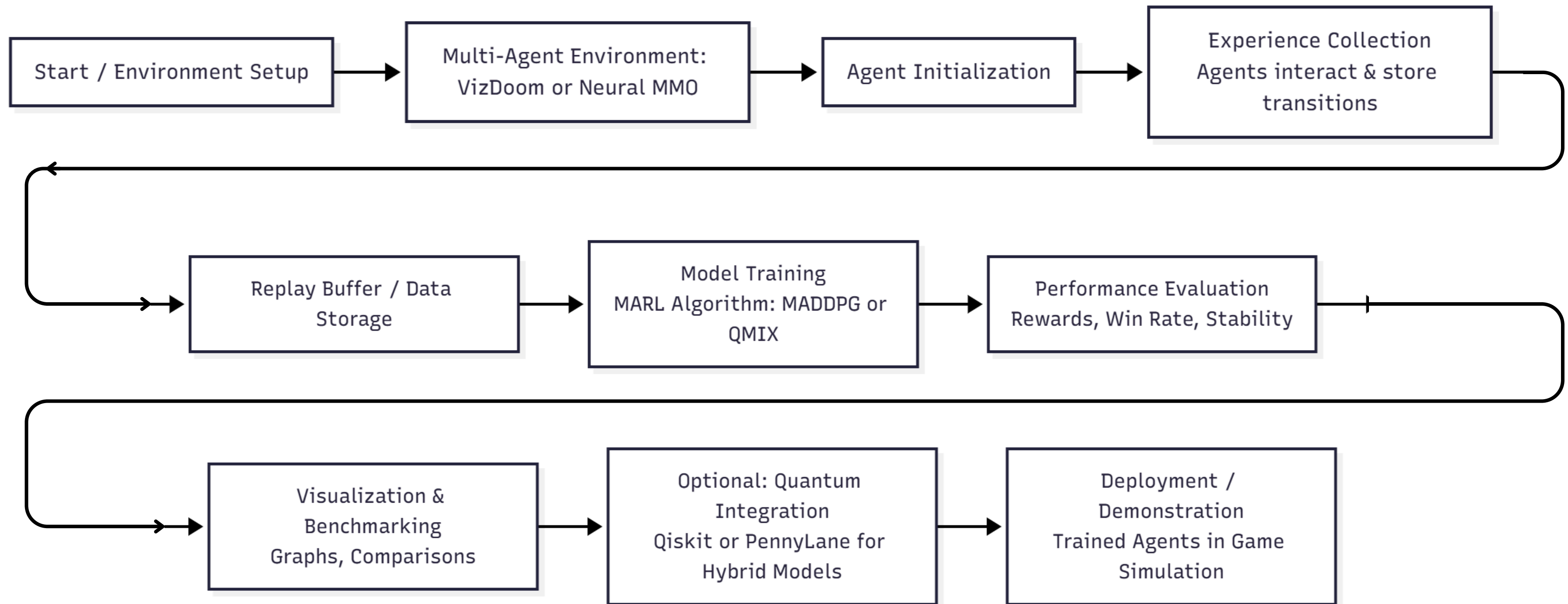
- State-action pairs are encoded into quantum states using parameterized quantum circuits (PQC) in Qiskit or PennyLane.
- The quantum layer outputs are used for policy updates in a hybrid neural architecture.

5. Deployment / Demonstration

- The trained agents are deployed in a live simulation to visualize cooperation or competition (e.g., Doom combat or resource collection in MMO).
- The system logs key statistics and renders gameplay videos.

Workflow Diagram

Go, change the world





Tools and Technologies

Go, change the world

Component	Tools / Libraries	Purpose
Game Environment	VizDoom / Neural MMO	VizDoom provides a 3D visual RL environment with first-person combat, ideal for competitive MARL. Neural MMO supports large-scale agent interactions in an open-world, ideal for emergent cooperation.
RL Frameworks	PettingZoo, Stable-Baselines3, Gymnasium	PettingZoo offers ready-made multi-agent environments and standardized APIs. Stable-Baselines3 provides reliable implementations of algorithms like DDPG, PPO, and MADDPG for stable training.
Deep Learning	PyTorch / TensorFlow	Used to design, train, and optimize neural networks that approximate agent policies and value functions. PyTorch is preferred for its flexibility and easier debugging.
Quantum Simulation (Optional)	Qiskit, PennyLane	Qiskit allows simulation of quantum circuits; PennyLane integrates quantum layers into PyTorch models. Used for hybrid quantum-classical experiments.
Visualization / Analysis	Matplotlib, Seaborn, TensorBoard	Used to plot training curves (reward vs. episodes, loss trends, win rates) and compare agent performance.
Hardware	NVIDIA GPU, IBM Quantum Simulator (optional)	GPUs accelerate neural training; IBM Quantum provides cloud-based quantum computing for QRL testing.



Timeline – Gantt Chart

Go, change the world



Expected Outcomes

- A functional prototype will be developed using Multi-Agent Reinforcement Learning (MARL) within an open-source environment such as VizDoom or Neural MMO. The system will demonstrate multiple agents learning to cooperate or compete autonomously through trial-and-error interactions. The prototype will include real-time visualization of agent decisions, reward progression, and performance metrics, effectively showcasing learned strategies and coordination behaviors.
- Performance will be evaluated using quantitative metrics such as average episodic reward, win rate, convergence speed, and policy stability. Comparative benchmarks will be drawn between classical MARL algorithms (e.g., MADDPG, QMIX) and hybrid quantum-integrated models.
- Future extensions include integrating real-time deployment of trained agents in interactive simulations, expanding to larger and more complex environments, and incorporating quantum reinforcement learning (QRL) for scalability testing. Further enhancements could involve adding communication learning between agents, transfer learning across environments, and real-world adaptation using robotic or IoT-based multi-agent platforms.

References

- [1] R. B. Ravish, S. Behera, A. Nayak, and B. Sahoo, “Optimization of Reinforcement Learning Using Quantum Computation,” *IEEE Access*, vol. 12, pp. 1–15, 2024.
- [2] T. Rashid, M. Samvelyan, C. Schroeder de Witt, J. Foerster, and S. Whiteson, “QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning,” in *Proc. ICML*, 2018, pp. 4292–4301.
- [3] J. Foerster, R. Y. Chen, M. Al-Shedivat, S. Whiteson, and P. Abbeel, “Learning with Opponent-Learning Awareness,” in *Proc. AAMAS*, 2018, pp. 122–130.
- [4] P. Peng, Q. Yuan, Y. Wen, et al., “Multiagent Bidirectionally-Coordinated Nets for Learning to Play StarCraft Combat Games,” *arXiv preprint arXiv:1703.10069*, 2017.
- [5] L. Busoniu, R. Babuška, and B. De Schutter, “A Comprehensive Survey of Multi-Agent Reinforcement Learning,” *IEEE Trans. Syst., Man, Cybern. C*, vol. 38, no. 2, pp. 156–172, 2008.
- [6] OpenAI, “OpenAI Five: Cooperative and Competitive Multi-Agent RL in Dota 2,” *OpenAI Blog*, 2019.
- [7] D. Silver, J. Schrittwieser, et al., “Mastering the Game of Go with Deep Neural Networks and Tree Search,” *Nature*, vol. 529, pp. 484–489, 2016.
- [8] J. Z. Leibo, V. Zambaldi, M. Lanctot, et al., “Multi-Agent Reinforcement Learning in Sequential Social Dilemmas,” in *Proc. AAMAS*, 2017, pp. 464–473.
- [9] D. Silver et al., “Mastering the Game of StarCraft II with Multi-Agent Reinforcement Learning,” *Nature*, vol. 575, pp. 350–354, 2019.
- [10] X. Wang, Z. Zhang, and W. Zhang, “Model-Based Multi-Agent Reinforcement Learning: Recent Progress and Prospects,” *J. AI Research*, 2022.
- [11] T. T. Nguyen, N. D. Nguyen, and S. Nahavandi, “Deep Reinforcement Learning for Multi-Agent Systems: A Review of Challenges, Solutions and Applications,” *arXiv preprint arXiv:1812.11794*, 2018.
- [12] Y. Yang and J. Wang, “An Overview of Multi-Agent Reinforcement Learning from a Game-Theoretical Perspective,” *Frontiers Comput. Sci.*, vol. 14, no. 3, pp. 533–547, 2020.
- [13] S. Y. C. Chen, “An Introduction to Quantum Reinforcement Learning (QRL),” *arXiv preprint arXiv:2409.05846*, 2024.
- [14] K. Zhang, Z. Yang, and T. Başar, “Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms,” *arXiv preprint arXiv:1911.10635*, 2019.
- [15] A. Bilal, M. U. K. Bangash, and M. Ali Jamshed, “Meta-Thinking in LLMs via Multi-Agent Reinforcement Learning: A Survey,” *arXiv preprint arXiv:2504.14520*, 2025.



**THANK YOU
All the BEST**